Experiment No 7

Name: Devang Deokule
Roll Number: A-58
TU ID: TU4S2425009

AIM: Implementation of any one clustering algorithm (K-means clustering) using JAVA.

Objectives:

After completing this experiment you will be able to:

1. Explain the differences among the two main styles of learning: supervised and unsupervised.
2. Implement simple algorithms for unsupervised learning.
3. Explain the problem of outlier on clustering algorithms.

LO's to be achieved:

LON	Lab Outcomes
LO4	Implement various data mining algorithms from scratch using languages like Python/ Java etc.

Theory:

The process of grouping a set of physical or abstract objects into classes of *similar* objects is called clustering. A cluster is a collection of data objects that are *similar* to one another within the same cluster and are *dissimilar* to the objects in other clusters. The grouping is accomplished by finding similarities between data according to characteristics found in the data itself. Thus clustering is viewed to be driven by the data itself and is often based on the similarity between attribute values.

Major Clustering Methods

Partitioning methods:

Given a database of n objects or data tuples, a partitioning method constructs k partitions of the data, where each partition represents a cluster and $k \leq n$. That is, it classifies the data into k groups, which together satisfy the following requirements:
(1) Each group must contain at least one object, and
(2) Each object must belong to exactly one group.
Notice that the second requirement can be relaxed in some fuzzy partitioning techniques.
To achieve global optimality in partitioning-based clustering would require the exhaustive enumeration of all of the possible partitions. Instead, most applications adopt one of a few popular heuristic methods, such as
(1) The *k-means* algorithm, where each cluster is represented by the mean value of the objects in the cluster, and
(2) The *k-medoids* algorithm, where each cluster is represented by one of the objects located near the center of the cluster.
These heuristic clustering methods work well for finding spherical-shaped clusters in small to medium-sized databases.

Hierarchical methods:
A hierarchical method creates a hierarchical decomposition of the given set of data objects. A hierarchical method can be classified as being either *agglomerative* or *divisive*, based on how the hierarchical decomposition is formed.

Density-based methods: Most partitioning methods cluster objects based on the distance between objects.. Other clustering methods have been developed based on the notion of *density*. Their general idea is to continue growing the given cluster as long as the density (number of objects or data points) in the "neighborhood" exceeds some threshold; that is, for each data point within a given cluster, the neighborhood of a given radius has to contain at least a minimum number of points.
DBSCAN and its extension, OPTICS, are typical density-based methods that grow clusters according to a density-based connectivity analysis.

K-Means Clustering:
The *k*-means algorithm takes the input parameter, *k*, and partitions a set of *n* objects into *k* clusters. The clusters are formed based on distance. The objects within a cluster are similar whereas the objects of different clusters are dissimilar. Cluster similarity is measured in regard to the *mean* value of the objects in a cluster, which can be viewed as the cluster's *centroid* or *center of gravity.*

$$E = \sum_{i=1}^{k} \sum_{p \in C_i} |p - m_i|^2,$$

where *E* is the sum of the square error for all objects in the data set; ***p*** is the point in space representing a given object; and ***mi*** is the mean of cluster *Ci* (both ***p*** and ***mi*** are multidimensional). In other words, for each object in each cluster, the distance from the object to its cluster center is squared, and the distances are summed. This criterion tries to make the resulting *k* clusters as compact and as separate as possible.

Initial values for the means are arbitrarily assigned. The convergence criteria could be based on the squared error or fixed number of iterations or when no (or very small) number of tuples is assigned to different clusters.

Algorithm: *k*-means.
The *k*-means algorithm for partitioning, where each cluster's center is represented by the mean value of the objects in the cluster.
Input:
k: the number of clusters,
D: a data set containing *n* objects.
Output: A set of *k* clusters.
Method:
(1) Arbitrarily choose *k* objects from *D* as the initial cluster centers;

(2) repeat
(3) (re)assign each object to the cluster to which the object is the most similar, based on the mean value of the objects in the cluster;
(4) update the cluster means, i.e., calculate the mean value of the objects for each cluster;
(5) until no change;

The method is relatively scalable and efficient in processing large data sets because the computational complexity of the algorithm is $O(nkt)$, where n is the total number of objects, k is the number of clusters, and t is the number of iterations.

Example:
Suppose that we are given the following items to cluster {2, 4, 10, 12, 3, 20, 30, 11, 25} And let k=2;

K-means clustering: Initially suppose the means m1= 2 and m2 =4; Using Euclidean distance we find that
k1 = { 2, 3}
k2 ={ 4,10,12,20,30,11,25}

M1	M2	K1	K2
2	4	2,3	10,11,12,20,,25 30
2.5	16	2,3,4	10,11,12,20,25,30
3	18	2,3,4,10	11,12,20,25,30
4.75	19.6	2,3,4,10,11,12	20,25,30
7	25	2,3,4,10,11,12	20,25,30

- K-means finds a local optimum and may actually miss the global optimum.
- K-means does not work on categorical data.
- K-means does not handle outliers well.

Conclusion:

In this experiment we have studied k-mean clustering method which is use partitioning based method. The K-means is the simplest method for clustering but the number of clusters must be given in advance and it is very sensitive to outliers.

Java Code

```
import java.util.*;
class k_means
        {
                static int count1,count2,count3;
                static int d[];
                static int k[][];
                static int tempk[][];
                static double m[];
                static double diff[];
                static int n,p;
        static int cal_diff(int a)
        {
                for(int i=0;i<p;++i)
                {
                if(a>m[i])
                        diff[i]=a-m[i];
                else
                diff[i]=m[i]-a;
            }
      int val=0;
        double temp=diff[0];
                for(int i=0;i<p;++i)
                {
                        if(diff[i]<temp)
                        {
                          temp=diff[i];
                          val=i;
                       }
                }//end of for loop
        return val;
       }


static void cal_mean()
{
        for(int i=0;i<p;++i)
                m[i]=0; // initializing means to 0
                int cnt=0;

                for(int i=0;i<p;++i)
                        {
                                cnt=0;
                                        for(int j=0;j<n-1;++j)
                                                {
                                                if(k[i][j]!=-1)
```

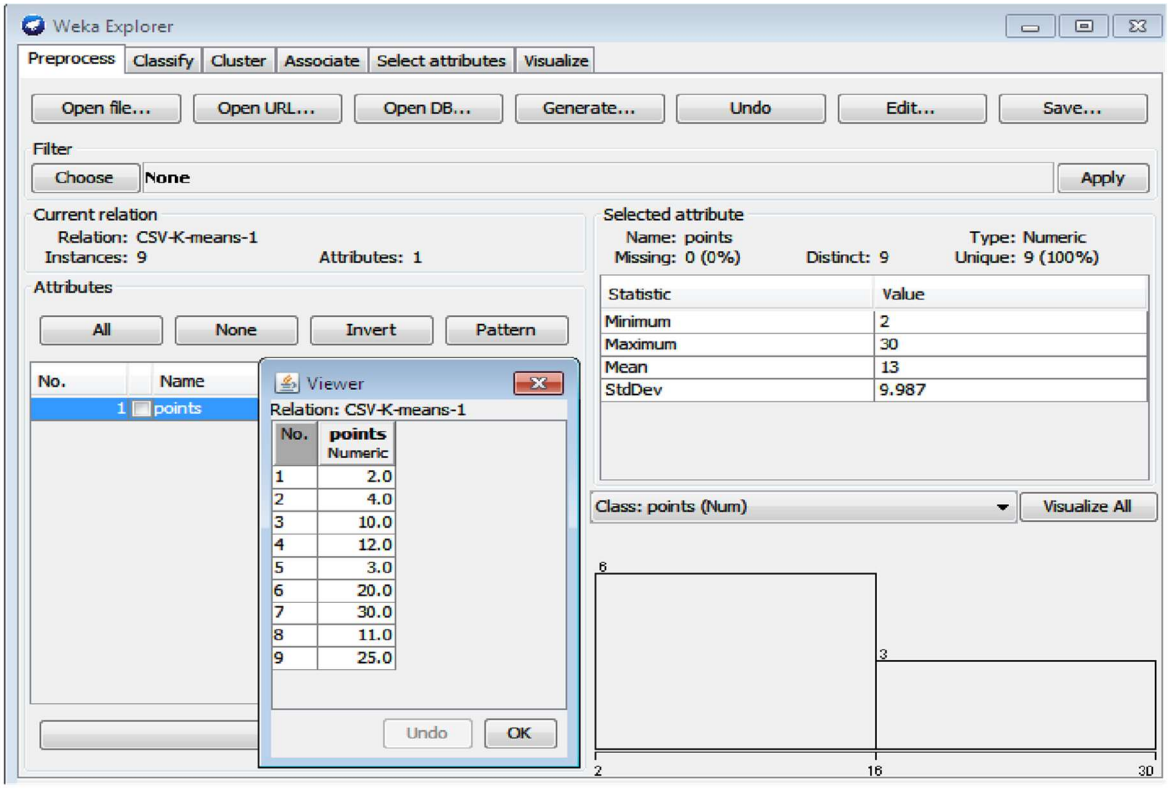
```
                                                    {
                                                        m[i]+=k[i][j];
                                                        ++cnt;
                                                    }
                                                }
                                m[i]=m[i]/cnt;
                            }
}
static int check1()
        {
         for(int i=0;i<p;++i)
           for(int j=0;j<n;++j)
                if(tempk[i][j]!=k[i][j])
                  {
                     return 0;
                  }
             return 1;
        }


public static void main(String args[])
{
        Scanner scr=new Scanner(System.in);
                /* Accepting number of elements */
                System.out.println("Enter the number of elements ");
                n=scr.nextInt();
                d=new int[n];
                /* Accepting elements */
                System.out.println("Enter "+n+" elements: ");
                for(int i=0;i<n;++i)
                        d[i]=scr.nextInt();
                        /* Accepting num of clusters */
                        System.out.println("Enter the number of clusters: ");
                        p=scr.nextInt();
                        /* Initialising arrays */
                        k=new int[p][n];
                        tempk=new int[p][n];
                        m=new double[p];
                        diff=new double[p];
                        /* Initializing m */
                        for(int i=0;i<p;++i)
                                m[i]=d[i];
                                int temp=0;
                                int flag=0;
                do
                  {
                    for(int i=0;i<p;++i)
                        for(int j=0;j<n;++j)
```

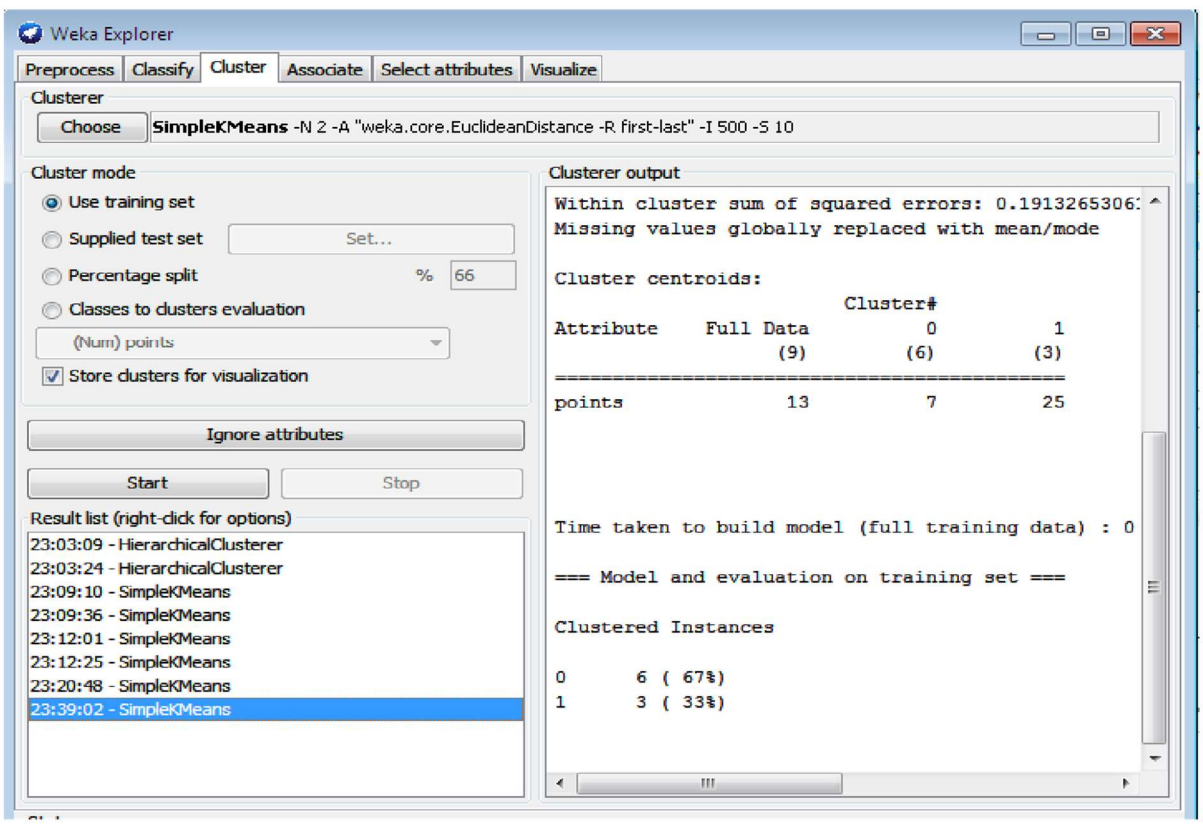
```
                    {
                    k[i][j]=-1;
                    }
                        for(int i=0;i<n;++i) // for loop will cal cal_diff(int) for every element.
                      {
                        temp=cal_diff(d[i]);
                        if(temp==0)
                              k[temp][count1++]=d[i];
                        else if(temp==1)
                               k[temp][count2++]=d[i];
                        else if(temp==2)
                                 k[temp][count3++]=d[i];
                    }


cal_mean(); // call to method which will calculate mean at this step.
flag=check1(); // check if terminating condition is satisfied. if(flag!=1)
/*Take backup of k in tempk so that you can check for equivalence in next step*/
        for(int i=0;i<p;++i)
        for(int j=0;j<n;++j)
        tempk[i][j]=k[i][j];
        System.out.println("\n\nAt this step");
        System.out.println("\nValue of clusters");
            for(int i=0;i<p;++i)
              {
                    System.out.print("K"+(i+1)+"{ ");
                      for(int j=0;k[i][j]!=-1 && j<n-1;++j)
                    {
                        System.out.print(k[i][j]+" ");
                        System.out.println("}");
                    }//end of for loop
                  System.out.println("\nValue of m ");
                 for(int i=0;i<p;++i)
                      System.out.print("m"+(i+1)+"="+m[i]+" ");
                       count1=0;count2=0;count3=0;
            }
       while(flag==0);
      {
          System.out.println("\n\n\nThe Final Clusters By Kmeans are as follows: ");
            for(int i=0;i<p;++i)
              {
                    System.out.print("K"+(i+1)+"{ ");
                    for(int j=0;k[i][j]!=-1 && j<n-1;++j)
                        System.out.print(k[i][j]+" ");
                        System.out.println("}");
            }
       }
}
```

Output:

```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows XP [Version 5.1.2600]
(C) Copyright 1985-2001 Microsoft Corp.

C:\Documents and Settings\student>cd desktop

C:\Documents and Settings\student\Desktop>set path="C:\Program Files\Java\jdk1.6
.0_10\bin"

C:\Documents and Settings\student\Desktop>javac k_means.java

C:\Documents and Settings\student\Desktop>java k_means
Enter the number of elements
6
Enter 6 elements:
5
10
23
27
36
39
Enter the number of clusters:
2


At this step

Value of clusters
K1{ 5 }
K2{ 10 23 27 36 39 }

Value of m
m1=5.0  m2=27.0

At this step

Value of clusters
K1{ 5 10 }
K2{ 23 27 36 39 }

Value of m
m1=7.5  m2=31.25

At this step

Value of clusters
K1{ 5 10 }
K2{ 23 27 36 39 }

Value of m
m1=7.5  m2=31.25


The Final Clusters By Kmeans are as follows:
K1{ 5 10 }
K2{ 23 27 36 39 }

C:\Documents and Settings\student\Desktop>_
```

Weka Output:


Weka Explorer
Preprocess
Classify
Cluster
Associate
Select attributes
Visualize
Open file...
Open URL...
Open DB...
Generate...
Undo
Edit...
Save...
Filter
Choose
None
Apply
Current relation
Relation: CSV-K-means-1
Instances: 9
Attributes: 1
Attributes
All
None
Invert
Pattern
No.
Name
1 points
Viewer
Relation: CSV-K-means-1
No. points Numeric
1 2.0
2 4.0
3 10.0
4 12.0
5 3.0
6 20.0
7 30.0
8 11.0
9 25.0
Undo
OK
Selected attribute
Name: points
Missing: 0 (0%)
Distinct: 9
Type: Numeric
Unique: 9 (100%)
Statistic
Value
Minimum 2
Maximum 30
Mean 13
StdDev 9.987
Class: points (Num)
Visualize All
6
3
2
16
30


Weka Explorer
Preprocess
Classify
Cluster
Associate
Select attributes
Visualize
Clusterer
Choose
SimpleKMeans -N 2 -A "weka.core.EuclideanDistance -R first-last" -I 500 -S 10
Cluster mode
Use training set
Supplied test set
Set...
Percentage split
% 66
Classes to clusters evaluation
(Num) points
Store clusters for visualization
Ignore attributes
Start
Stop
Result list (right-click for options)
23:03:09 - HierarchicalClusterer
23:03:24 - HierarchicalClusterer
23:09:10 - SimpleKMeans
23:09:36 - SimpleKMeans
23:12:01 - SimpleKMeans
23:12:25 - SimpleKMeans
23:20:48 - SimpleKMeans
23:39:02 - SimpleKMeans
Clusterer output
Within cluster sum of squared errors: 0.1913265306
Missing values globally replaced with mean/mode
Cluster centroids:
Cluster#
Attribute Full Data 0 1
(9) (6) (3)
points 13 7 25
Time taken to build model (full training data) : 0
=== Model and evaluation on training set ===
Clustered Instances
0 6 (67%)
1 3 (33%)

Post Lab Exercise:

Qu-1: Suppose that the data mining task is to cluster points (with (x, y) representing location) into three clusters, where the points are

A1(2, 10),A2(2, 5),A3(8, 4),B1(5, 8),B2(7, 5),B3(6, 4),C1(1, 2),C2(4, 9).

The distance function is Euclidean distance. Suppose initially we assign A1, B1, and C1 as the center of each cluster, respectively. Use the k-means algorithm to show only

(a) the three cluster centers after the first round of execution.

Answer:

After the first round, the three new clusters are: (1) {A1}, (2) {B1,A3,B2,B3,C2}, (3) {C1,A2}, and their centers are (1) (2, 10), (2) (6, 6), (3) (1.5, 3.5).

(b) the final three
clusters. Answer:

The final three clusters are: (1) {A1,C2,B1}, (2) {A3,B2,B3}, (3) {C1,A2}.